Lecture 11: Brief Overview of Scala for Spark

DSL351 / DSP352

Amit Kumar Dhar

ED1, 406A
amitkdhar@iitbhilai

February 2, 2026

Introduction to Scala

What is Scala?

- **Sca**lable **La**nguage.
- A modern multi-paradigm programming language designed to express common programming patterns in a concise, elegant, and type-safe way.
- Integrates features of **Object-Oriented** and **Functional** languages.
- Runs on the **Java Virtual Machine (JVM)**.
- Fully interoperable with Java.

Why Scala for Spark?

- Apache Spark is written in Scala.
- Scala provides the most complete and up-to-date API for Spark.
- **Conciseness:** Scala code is often 2-3x shorter than equivalent Java code.
- **Functional Nature:** Fits perfectly with the MapReduce/Spark distributed processing model (map, filter, reduce).
- **Type Safety:** Catch errors at compile time rather than runtime.

Scala vs. Java: Comparison

Feature	Java	Scala
Paradigm	Object-Oriented (mostly)	OO + Functional
Verbosity	High (Boilerplate)	Low (Concise)
Type System	Static	Static + Type Inference
Semicolons	Mandatory	Optional
Variables	Mutable by default	Immutable preferred
Functions	Methods	First-class citizens

Scala vs. Java: Code Comparison

Java (POJO)

```java
public class Person {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }
    // Getters, Setters, toString...
}
```

Scala (Case Class)

```scala
case class Person(name: String, age: Int)
// Includes constructor, getters,
// toString, equals, hashCode automatically
```

The Scala REPL

- Interactive shell for running Scala code (Read-Eval-Print Loop).
- Great for testing small snippets and learning APIs.
- Start by typing `scala` in your terminal (if installed).

```
$ scala
Welcome to Scala 2.12.15 ...
scala> val x = 10
x: Int = 10

scala> x * 2
res0: Int = 20
```

Core Syntax & Concepts

Variables: val vs. var

Immutability is key in Distributed Computing.

- `val`: Immutable reference (Constant). **Preferred**.
- `var`: Mutable reference (Variable). Avoid unless necessary.

```
val msg = "Hello World"
// msg = "Goodbye"  // Error: Reassignment to val

var count = 0
count = 1           // OK
```

Basic Data Types

Scala is purely object-oriented. Even primitives are objects.

- `Int, Double, Float, Long, Short, Byte`
- `Boolean`
- `Char`
- `String` (from Java)
- `Unit` (Correspond to `void` in Java)

```
1 val a: Int = 5
2 val b: Double = 3.14
3 val c: Boolean = true
```

Type Inference

Scala compiler is smart enough to infer types. You don't always have to declare them.

```scala
val number = 42              // Infers Int
val name = "Spark"           // Infers String
val isActive = true          // Infers Boolean

// Explicit type still allowed/recommended for public APIs
val salary: Double = 50000.0
```

Functions

Functions are expressions that return the last value. `return` keyword is rarely used.

```
// Standard function definition
def add(x: Int, y: Int): Int = {
    x + y  // Implicit return
}

// Single line syntax
def multiply(x: Int, y: Int): Int = x * y

println(add(5, 3))       // Output: 8
```

Anonymous Functions (Lambdas)

Crucial for Spark transformations (map, filter).

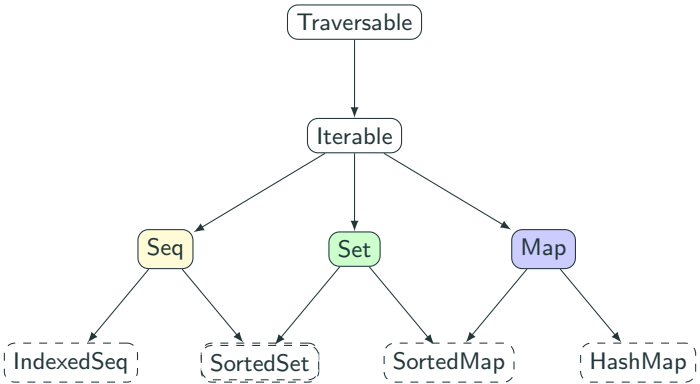
```
// Function literal
(x: Int) => x + 1

// Assigning to a variable
val increment = (x: Int) => x + 1

println(increment(10)) // 11
```

Collections & Data Structures

Scala Collections Hierarchy



Mutable vs. Immutable

- **Immutable (Default):** `scala.collection.immutable`. Modifications create a *new* collection. Safe for concurrency.
- **Mutable:** `scala.collection.mutable`. Modifications update in-place.

```
// Immutable List (Default)
val list1 = List(1, 2, 3)
val list2 = 0 :: list1  // Creates new list (0, 1, 2, 3)

import scala.collection.mutable.ArrayBuffer
val buf = ArrayBuffer(1, 2)
buf += 3                // Updates buf to (1, 2, 3)
```

Sequences: List, Vector, Array

```
// List: Linked List (Good for head access/prepend)
val nums = List(1, 2, 3, 4)
val head = nums.head    // 1
val tail = nums.tail    // List(2, 3, 4)

// Vector: Indexed Sequence (Good for random access)
val vec = Vector(1, 2, 3, 4)
val elem = vec(2)       // 3

// Array: Mutable, fixed size (Java Array wrapper)
val arr = Array(1, 2, 3)
arr(0) = 10             // Updates in place
```

Sets and Maps

```scala
// Set: Unique elements, unordered
val fruits = Set("apple", "banana", "apple")
// fruits is Set("apple", "banana")

// Map: Key-Value pairs
val codes = Map("US" -> "United States", "IN" -> "India")

println(codes("IN"))            // "India"
println(codes.getOrElse("UK", "Unknown")) // Safe access
```

Tuples

Immutable containers for heterogenous data. Fixed size.

- Useful for returning multiple values from a function.
- Spark RDDs often use tuples for (Key, Value) pairs.

```
val t = (1, "Hello", true)

println(t._1)  // 1
println(t._2)  // "Hello"
println(t._3)  // true

// Pattern matching extract
val (id, msg, status) = t
```

Option Type: No more NullPointerException

`Option[T]` represents a value that may or may not exist.

- `Some(value)`: The value exists.
- `None`: The value is missing.

```
val scores = Map("Alice" -> 90, "Bob" -> 85)

val aliceScore: Option[Int] = scores.get("Alice") // Some(90)
val charlieScore: Option[Int] = scores.get("Charlie") // None

val finalScore = charlieScore.getOrElse(0) // 0
```

Data Modelling

Classes and Objects

Class

- Blueprint for creating objects.
- Can contain methods and state.

Object (Singleton)

- A class with exactly one instance.
- Used for static members/methods.
- Entry point (`main`) lives here.

```
class Greeter(prefix: String) {
    def greet(name: String): Unit = println(s"$prefix $name")
}
object MainApp {
    def main(args: Array[String]): Unit = {
        val g = new Greeter("Hello")
        g.greet("World")
    }
}
```

Case Classes

Ideal for modelling immutable data records. Used extensively in Spark DataFrames/Datasets.

```
// One line definition
case class Transaction(id: Int, amount: Double, currency: String)

// Usage (No 'new' keyword needed)
val t1 = Transaction(101, 50.0, "USD")
val t2 = Transaction(102, 75.0, "EUR")

println(t1.amount)  // 50.0
```

Case Class Features

Why are they special?

1. **Immutable fields:** Parameters are `val` by default.
2. **Factory Method:** `apply` method generated (no `new`).
3. **Equality:** `equals` and `hashCode` based on fields, not reference.
4. **Copying:** `copy` method creates modified clones.
5. **Serializable:** Ready for network transmission (Spark).
6. **Pattern Matching:** Support decomposability.

Pattern Matching

Like `switch` in Java, but much more powerful.

```scala
val x: Any = "Hello"

x match {
    case 1 => println("One")
    case "Hello" => println("Greeting")
    case y: Int => println("Other Integer: " + y)
    case _ => println("Something else")
}

// Matching Case Classes
val t = Transaction(101, 50.0, "USD")
t match {
    case Transaction(_, amt, "USD") if amt > 100 => println("High Value USD")
    case Transaction(_, _, curr) => println(s"Currency: $curr")
}
```

Functional Programming

First-Class Functions

- Functions are values.
- Can be passed as arguments to other functions.
- Can be returned from functions.
- **Higher-Order Functions (HOF):** Functions that take other functions as parameters (e.g., `map`, `filter`).

Common HOFs on Collections

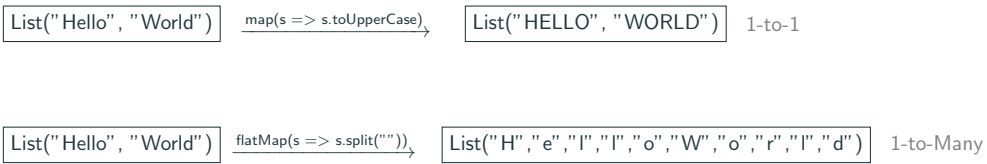
```scala
val numbers = List(1, 2, 3, 4, 5)

// Map: Transform each element
val squares = numbers.map(x => x * x) // List(1, 4, 9, 16, 25)

// Filter: Select elements
val evens = numbers.filter(x => x % 2 == 0) // List(2, 4)

// Reduce: Aggregate
val sum = numbers.reduce((a, b) => a + b) // 15
```

Visualizing Map vs. FlatMap



- **map:** Returns one element for each input element.
- **flatMap:** Returns a sequence for each input, then flattens results.

Immutability & Pure Functions

- **Pure Function:** Output depends *only* on input. No side effects (no printing, no var mutation, no external I/O).
- **Benefit:** Easier to reason about, test, and **parallelize**.

```
// Pure
def add(a: Int, b: Int): Int = a + b

// Impure (Side Effect)
var total = 0
def addToTotal(a: Int): Unit = { total += a }
```

Expression Oriented

Everything in Scala is an expression that returns a value.

```scala
val x = 10

// If-else returns a value
val result = if (x > 5) "Big" else "Small"

// Block returns last line
val y = {
    val a = 5
    val b = 2
    a * b  // Returns 10
}
```

Lazy Evaluation

Strict vs. Lazy

- **Strict (Eager):** Value computed immediately when variable is defined. (Java default, Scala `val`).
- **Lazy:** Computation deferred until the value is actually accessed.
- Crucial for Spark's performance optimization (Transformation chaining).

The lazy val keyword

```scala
// Executed immediately
val eager = {
    println("I am eager")
    10
}

// Executed only when 'lazyVal' is used
lazy val lazyVal = {
    println("I am lazy")
    20
}

println("Start")
println(lazyVal) // Triggers computation now
println(lazyVal) // Uses cached value (no print)
```

Lazy Streams (Infinite Collections)

Since elements are computed on demand, we can define infinite sequences.

```
// Stream of integers starting from 1
def naturals(n: Int): Stream[Int] = n #:: naturals(n + 1)

val stream = naturals(1)

// Takes top 5, computes only those 5
println(stream.take(5).toList)
// Output: List(1, 2, 3, 4, 5)
```

Questions?